

MCU Interrupts & Task Scheduling

1. System Design Architectures (Polling vs. Interrupts)

An embedded system's efficiency is determined by how fast it responds to external events. There are two primary software architectures:

Polling (Super Loop)

The application runs inside an infinite **while(1)** loop, sequentially checking peripherals (e.g., switches, ADC) to see if an event occurred.

✓ Pros

- Highly predictable — you always know exactly what the CPU is executing at any given microsecond.
- Doesn't require complex hardware like an interrupt controller.

✗ Cons

- Wastes massive amounts of CPU time checking idle peripherals.
- Suffers from high latency and can miss quick events (e.g., a fast button press/release while the CPU is busy elsewhere).

Interrupts (Foreground / Background)

The system is split into a **Background** (the continuous main program) and a **Foreground** (Interrupt Service Routines — ISRs).

✓ Pros

- Immediate response to events without wasting CPU time.
- Lower power consumption, and zero chance of missing quick events.

✗ Cons

- Execution time becomes unpredictable — an interrupt can preempt the main code at any moment.
- Requires extra hardware (like the GIC/NVIC).

2. Interrupt Firing Conditions

For an MCU to respond to an interrupt, three critical conditions must be logically active at the same time — acting like an **AND gate**:

Global Interrupt Enable (GIE)	The master switch allowing the CPU to accept interrupts.
Peripheral Interrupt Enable	Specific permission for a peripheral (e.g., UART or Timer) to generate an interrupt.
Interrupt Flag	The actual hardware signal raised when the event occurs (e.g., End of Conversion).

3. Interrupt Handling: Vectored vs. Non-Vectored

Vectored Interrupts (Fast & Fixed)

- The hardware dictates fixed priorities.
- The MCU jumps directly to a specific address in a hardware "Vector Table" dedicated to that exact interrupt.
- The Vector Table contains a jump instruction straight to the corresponding ISR.

Non-Vectored / Flexible Interrupts (Slower & Software-based)

- All interrupts send the CPU to one common ISR address.
- Inside this ISR, the programmer writes if/else software conditions to check which flag was raised.
- This determines priority via software — but is highly inefficient if the system has hundreds of interrupts.

4. Vector Tables and Startup Code

The location of the Vector Table is crucial and comes in two forms:

Static (in Flash/ROM)	Burned by the programmer. Safe, read-only, and ready to handle interrupts the moment the MCU powers on.
Dynamic (in RAM)	Initialized by the Startup Code.

■ The Danger

If an interrupt fires while the Startup Code is still filling the RAM table, the CPU will fetch "garbage values" and jump to an undefined address, crashing the system.

■ The Solution

The Startup Code must disable the Global Interrupt (GIE) initially, populate the table, and only enable interrupts before jumping to `main()`.

Main duties of Startup Code

- Load **.data** from Flash to RAM.
- Allocate / zero out **.bss** in RAM.
- Initialize the Vector Table (if dynamic).
- Initialize the Stack Pointer.
- Branch to **main()**.

5. Interrupt Nesting

Normal Nesting	A higher-priority interrupt preempts a currently running lower-priority ISR. Acceptable if the design supports it.
Self-Nesting	An ISR gets interrupted by itself repeatedly — a catastrophic design flaw that leads to Stack Overflow.

■ Prevention

Hardware automatically disables the GIE when entering an ISR, and re-enables it upon exit, ensuring the ISR finishes without being preempted by an interrupt of equal or lower priority.

6. Clearing the Interrupt Flag

If the raised flag is not cleared to zero, the CPU will get stuck in an infinite loop executing the same ISR. It is cleared in 4 possible ways depending on the MCU architecture:

- Automatically by hardware when entering or exiting the ISR.
- By writing 0 to the register.
- By writing 1 to the register (in some architectures).
- Simply by reading a specific register.

■ Note

If using the Polling method, the programmer must manually clear the flag in the software.

7. Types of Interrupts

Hardware Interrupts (Asynchronous)	Triggered by external peripherals. Noticeable latency since the signal must travel through the interrupt controller.
Software Interrupts / Exceptions (Synchronous)	Triggered internally by the CPU (e.g., dividing by zero, invalid instruction). Near-zero latency.

Interrupts are further classified into:

- **Maskable** — can be disabled via the GIE.
- **Non-Maskable (NMI)** — like the Reset button, which executes unconditionally regardless of the GIE state.

8. Context Switching (Step-by-Step)

When an interrupt preempts the main code, the MCU must carefully pause and save its state:

- 1 Complete the current assembly instruction.
- 2 Hardware disables Global Interrupts.
- 3 Clear the interrupt flag.
- 4 **Context Save (Push to Stack):** Hardware saves the Program Counter (PC) and Status Registers (like PSW); software pushes the General Purpose Registers.
- 5 Jump to the Vector Table, then branch to the ISR.
- 6 **Context Restore (Pop from Stack):** Reverse the process by popping registers from the Stack.
- 7 Enable Global Interrupts via software right before exiting.
- 8 Resume the main program.

■ Difference from Function Calls

Function calls are expected, so the compiler handles all context saving via software. Interrupts are sudden, so the hardware must automatically step in to save the critical context.

9. Timing Terminology

Response Time	The fixed hardware time required to prepare the context and jump to the Vector Table.
Latency	The total time from the event occurring until the ISR starts executing. Latency = Response Time + Waiting Time (if a higher priority ISR is already running).
Recovery Time	The time spent unstacking (restoring context) and returning to the main application.

10. Interrupt Overload

If peripherals fire interrupts faster than the CPU can process them, the CPU load reaches 100%. The system gets entirely trapped in the ISRs and never returns to the background main code.

Solutions

- Implementing software schedulers to discard rapidly repeating interrupts.
- Using hardware timers inside the Interrupt Controller to throttle the interrupt frequency.

— End of Document —